```
      {name: "Project Runway", tasks: "Choose Fabric:3\nMake it Work:5"}})
    @project = Project.find_by(name: "Project Runway")
    follow_redirect!
    assert_select("#project_#{@project.id} .name", "Project Runway")
    assert_select("#project_#{@project.id} .total-size", text: "8")
  end

  test "does not allow a user to create a project without a name" do
    post(projects_path, params: {project:
      {tasks: "Choose Fabric:3\nMake it Work:5"}})
    assert_select(".new_project")
  end

  test "behaves correctly with a database failure" do
    workflow = stub(success?: false, create: false, project: Project.new)
    CreatesProject.stubs(:new)
      .with(name: "Project Runway",
            task_string: "Choose Fabric:3\nMake it Work:5")
      .returns(workflow)
    post(projects_path, params: {project:
      {name: "Project Runway", tasks: "Choose Fabric:3\nMake it Work:5"}})
    @project = Project.find_by(name: "Project Runway")
    assert_select(".new_project")
  end
end
```

There are a few differences between these tests and the system tests you wrote before.

Rather than simulating filling out the form with Capybara, you're sending arguments through the Rails routing table directly using the post helper. This is the same method taking the same arguments as RSpec request specs, as discussed in Chapter 11, *Testing Rails Display Elements*, on page 217.

In the first test, the successful action of the controller is a redirect. Rails integration tests don't follow redirects by default, so use follow_redirect! to force the test to continue so that you can check the output.

Rails integration tests use assert_select to check output, which is similar to the Capybara have_selector matcher you've already seen, but is a separate implementation with its own syntax quirks. By default assert_select works against the test's response body.

The selector syntax that assert_select uses is very similar to jQuery and other DOM selection tools, though assert_select uses its own HTML parser. As in jQuery, the use of two separate selectors means you'd expect to match an instance of the first selector, which contains an instance of the second. For example, you're looking for an HTML element with the DOM ID of the form